

OS : interface entre hardware et user

Kernel : composant de l'os qui fournit une interface entre le matériel et les autres composants logiciels de l'os. Il initialise les autres composants de l'os

Exemple, une touche de clavier pressée > passe par le kernel

Boot loader : logiciel indépendant de l'OS, historiquement stocké dans les 512 premiers octets de la mémoire non volatile d'un ordinateur (donc d'un disque)

Il charge le kernel dans la mémoire volatile/RAM pour démarrer le système (exemples : winboot, grub)

Pour créer un dual boot, il est préférable d'installer d'abord windows pour que winboot n'écrase pas un autre bootloader > est ce qu'il est possible de ne pas installer winboot grâce à un autounattend etc ?

Le bootloader est chargé en RAM par le BIOS

Basic Input/Output System (maintenant Unified Extented Firmware Interface), logiciel indépendant de l'OS stocké dans de la mémoire ROM (Read Only Memory) sur la CM

Note : la ROM est techniquement réécritable, en EEM (Electrically Erasable Memory)

C'est le premier programme démarré après la mise sous tension de l'ordinateur, il initialise et teste le matériel, localise et met en mémoire le bootloader puis le démarre.

Les BIOS UEFI sont aussi stockés sous la forme d'un fichier .efi sur une partition spéciale d'un disque dur appelé EFI System Partition (ESP), qui permet plus de modularité et de fonctionnalités qu'un BIOS de base

Typiquement, les modifications matérielles proposées par le mode sans échec de windows est permis par l'UEFI

Dans un ordinateur doté d'un UEFI, le Boot Loader est installé sur la partition ESP

A la fin du boot, le Kernel démarre le gestionnaire de services, qui démarrera par la suite tout le reste du système d'exploitation (y compris les CLI ou GUI)

Date importante : 1969, premier système UNIX

1981 :MSDOS (MicroSoft Disk Operating System), 1^{er} OS commercial de Microsoft pour les PC, basé sur l'Architecture IBM x86 (pc IBM 8086)

1985 : Windows 1.0, premier système Microsoft doté d'une GUI (basé sur le kernel MSDOS)

1989 : NeXTSTEP OS doté d'une GUI et un noyau open source dérivé d'Unix (NextOS) développé par Steve Jobs. Raté commercial mais OS très bon, racheté par apple, utilisé comme base pour MacOS

1991 : Linux (inspiré d'Unix) développé par Linus Torvald. Logiciel libre. Il a été poussé par l'arrêt de la distribution de la version étudiante d'Unix par AT&T. Distribué par GNU (Richard Stallman), sous les termes de la GNU General Public Licence (tt comme Blender, Gimp et d'autres ajd)

Aujourd'hui

Windows n'est plus basé MS-DOS depuis 2001. Le noyau de Windows est propriétaire

MacOS est basé sur un kernel dérivé d'Unix, le noyau XNU, qui est libre. Ce sont les autres composants de l'OS sont principalement des logiciels propriétaires (c'est pourquoi apple ne peut pas vendre MacOS, mais uniquement la licence d'utilisation)

Le noyau Linux est utilisé par de nombreux OS (Debian, Android, Ubuntu, Mint....) Les OS basés sur Linux peuvent être libres de droit ou en partie propriétaires.

Noyau et Pilotes

Kernel > invisible à l'utilisateur, il permet aux programmes d'être exécutés (en leur « exposant » de la mémoire vive)

Il gère les événements produits au niveau de la couche matérielle (appelés 'interruptions') qui déclenchent l'exécution d'un bout de code (ex : touche du clavier, clic souris, périphérique branché, fin d'écriture etc...)

Il gère aussi les événements produits au niveau couche logicielle, les « appels système »

Il fait l'interface entre matériel et logiciel, et gère les accès de ces couches aux ressources (matériel)

Le noyau est chargé en mémoire sous forme de code machine. Les codes machines x86 (pour les archis 32 bits) et x64 (pour les archis 64 bits) sont utilisés sur les matériels de type PC compatibles

On distingue deux familles de noyaux : SISC et RISC

Les processeurs SISC sont l'architecture historique

Les processeurs « risc » ou ARM, exécutant des tâches plus simples, sont par conséquent moins coûteux en énergie, sont l'architecture plus récente, répandue notamment dans tout ce qui est téléphone

Les codes machines ne sont pas compatibles entre les architectures

Les codes machines sont généralement écrits en C, puis transformés en code machine correspondant aux différents noyaux, grâce à des compilateurs. Les compilateurs vont aussi ajouter des surcouches de compatibilité avec le noyau sur lequel ils ont été exécutés (adresses mémoires etc) d'où la nécessité de compiler sur une architecture noyau similaire, ou sinon d'ajouter une « couche de compatibilité » qui permet de traduire les instructions noyau (compatibilité x86 et x64)

Pour avoir une compatibilité x86-ARM, il faut passer par de l'émulation, l'émulateur faisant l'interface pour traduire les instructions de code du logiciel émulé pour le traduire en code machine compatible avec l'architecture utilisée.

Lorsque l'on va développer une application (en C par exemple), on utilise des bibliothèques standards (écrites en C) qui contiennent des listes de fonction code, et les adresses mémoires ou codes machines qui y correspondent pour faire comprendre et exécuter les tâches par le noyau

Les bibliothèques standards sont fournies avec les OS. Les OS dérivés UNIX (hors windows donc) suivent la norme IEEE1003 / POSIX

2 catégories de noyau pour les systèmes UNIX-like

1 : Microkernel, gère la communication entre les programmes chargés en RAM en utilisant de l'Inter Process Communication (permet d'utiliser la RAM pour la lecture/écriture pour ignorer la localisation physique des données), permet de la résilience très forte de l'OS et de ses fonctionnalités de base.

Problème de ces architectures : limite le développement d'un écosystème de logiciel, car bcp de versions alternatives de système (chaque partie du système étant gérée par des composants séparés, et par tt par le kernel)

Historiquement, c'était le cas chez apple, d'où le fait qu'il y avait peu de développement indépendant pour système apple

2 : Noyau monolithique, le noyau inclut aussi la gestion des pilotes, le dispatcher (répartition de la charge de travail), le système de fichier. L'avantage est d'avoir un dataset d'appels système bien défini, pour que le développement de logiciels faisant appel à plusieurs composants système soit plus simple

Actuellement, on ne trouve plus que Minix comme OS basé sur un système microkernel
osdev.org permet de tester la création d'un OS (basique)

Pilotes

Les pilotes de périphériques sont des blocs de code qui font :

- Partie du noyau

- Ou sont insérés dynamiquement dans un noyau en cours d'exécution sous la forme de « modules »

Les pilotes ajoutent au noyau :

- La gestion des interruptions entre les périphériques matériels et le noyau

- La gestion d'appels systèmes entre des applications et le noyau

Ils sont soit fournis par le constructeurs du matériel soit développés par des membres de la communauté contribuant au développement de l'OS

Ils peuvent être sous licence propriétaire ou open source

Sur Linux, la liste des modules actuellement chargés s'obtient avec `lsmod`

De nouveaux modules s'ajoutent avec `modprobe <file>`

Pour les infos relatives à un module : `modinfo <module name>`